Calculadora de Edad en Python y C++: Una Comparación de Paradigmas de Programación

Malek Cheheid

Escuela Superior Técnica N.º 1 «Eduardo Ader»
Vicente López, Buenos Aires, Argentina

Laboratorio de Programación (LPR)
5.º Año — 3.º A

Docente: Yamil Gandgulia

1.º Cuatrimestre 2026

Julio de 2026

Repositorio:
https://github.com/malekcheheid-art/calculadoraedad.git

Resumen

El presente informe documenta el desarrollo de un programa que calcula la edad exacta de una persona a partir de su fecha de nacimiento, validando que dicha fecha exista realmente en el calendario. La actividad consistió en resolver un mismo problema algorítmico en dos lenguajes de programación con características opuestas: Python, un lenguaje de alto nivel con tipado dinámico, y C++, un lenguaje estructurado de bajo nivel con tipado estático. Se aplicó el concepto de *filtro de consistencia* para descartar fechas imposibles (días 35, meses 13, años 20900) y se implementó la regla del año bisiesto, incluyendo el caso secular de los años divisibles por 400. Se documenta además la comparación sistemática entre ambas implementaciones y se presentan los resultados de diecinueve (19) casos de prueba ejecutados. El programa cumple con la especificación en su totalidad.

Palabras clave: programación estructurada, Python, C++, algoritmo, validación de datos, año bisiesto

1. Introducción

La resolución de problemas mediante programación constituye uno de los objetivos centrales de cualquier curso introductorio de informática. En esta actividad se seleccionó un problema de relevancia cotidiana: determinar la edad exacta de una persona a partir de su fecha de nacimiento. Aunque el cálculo aparente es simple, esconde dos dificultades que lo hacen apropiado para una revisión: determinar *cuándo* se considera ocurrido un cumpleaños, y determinar *qué fechas existen* en el calendario. Por indicación de la cátedra, el mismo algoritmo se implementó en dos lenguajes de programación elegidos precisamente porque difieren en su modelo de ejecución subyacente: Python, un lenguaje interpretado de alto nivel con tipado dinámico, y C++, un lenguaje compilado y estructurado de tipado estático.

La idea que estuvo detrás del trabajo fue ver cómo se nota, en la práctica, que el *algoritmo* es invariante entre lenguajes pero su *expresión* no lo es: una misma condición

lógica exige una cantidad distinta de construcciones, un tratamiento distinto de los tipos y estrategias diferentes para responder a entradas erróneas en cada lenguaje. Mostrar ese contraste fue, en definitiva, el objetivo del ejercicio.

2. Objetivos

2.1. Objetivo general

Implementar y comparar un algoritmo de cálculo de edad exacta con validación de calendario, desarrollado en Python y en C++, aplicando los conceptos de lógica, algoritmos y estructuras de control revisados en el cursado.

2.2. Objetivos específicos

1. Determinar la edad restando el año de nacimiento al año actual.
2. Descontar un año cuando la persona aún no haya cumplido anios en la fecha de referencia.
3. Validar la fecha ingresada mediante un filtro de consistencia que rechace fechas inexistentes en el calendario real.
4. Implementar correctamente la regla del año bisiesto, incluidos los casos seculares (divisibles por 100 y por 400).
5. Resolver la misma lógica en Python y en C++.
6. Identificar y documentar las diferencias entre ambos lenguajes.
7. Verificar el funcionamiento mediante un conjunto de casos de prueba.

3. Marco teórico

3.1. Filtro de consistencia

Un *filtro de consistencia* es el conjunto de condiciones lógicas que el programador incorpora para impedir que el usuario introduzca datos imposibles en el sistema. En este problema el filtro debe garantizar que la fecha ingresada exista en el calendario real, lo que implica verificar cuatro familias de restricciones: el rango del año, el rango del mes, el

rango general del día y, finalmente, la cantidad de días del mes particular.

3.2. Año bisiesto

El año bisiesto es aquel que tiene 366 días en lugar de 365, por lo que febrero posee 29 días. La regla que lo determina es la siguiente expresión lógica:

```
(anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0)
```

La regla tiene tres partes. Primero, el año debe ser divisible por 4. Segundo, las excepciones: los años divisibles por 100 (por ejemplo, 1900) *no* son bisiestos, porque 100 no es divisible por 400. Tercero, la excepción a la excepción: los años divisibles por 400 (por ejemplo, 2000) sí lo son. Este caso secular es precisamente el que distingue una implementación correcta de una aproximada.

4. Metodología

El trabajo se desarrollo en cuatro etapas. En primer lugar, se analizó el enunciado y se identificaron los datos de entrada (día, mes y año de nacimiento) y la fecha de referencia (8 de julio de 2026). En segundo lugar, se diseñó el algoritmo en pseudocódigo, verificando que el cálculo de la edad se realizara en el orden adecuado: primero la resta de años, luego la corrección por cumpleaños, y por último la validación. En tercer lugar, se implementó el algoritmo en cada lenguaje respetando el esqueleto (scaffolding) provisto por la cátedra. En cuarto lugar, se construyó y ejecutó una batería de diecinueve (19) casos de prueba que cubren los límites del calendario, los casos críticos de cumpleaños y las entradas no numéricas. Respecto del entorno de ejecución, la versión en Python se ejecutó localmente con Python 3.14, mientras que la versión en C++ se compiló y ejecutó mediante el compilador en línea OnlineGDB, dado que el equipo no disponía de un compilador local instalado. Este contraste de entornos quedó registrado mediante capturas de pantalla, conforme a lo solicitado por la cátedra.

4.1. Algoritmo en pseudocódigo

```
INICIO
  LEER diaN, mesN, anioN
  diaA <- 8 ;  mesA <- 7 ;  anioA <- 2026
```

```
  SI (esFechaValida(diaN, mesN, anioN) = FALSO) ENTONCES
        ESCRIBIR "La fecha ingresada no es valida"
  SINO
        edad <- anioA - anioN

        // Correccion: solo si ya cumplio anios en el mes o dia actual
        SI (mesN > mesA) O ((mesN = mesA) Y (diaN > diaA)) ENTONCES
              edad <- edad - 1
        FIN SI

        ESCRIBIR "Edad calculada: " , edad
  FIN SI
FIN

// Subalgoritmo: anio bisiesto
FUNCION esBisiesto(anio)
  RETORNAR ((anio MOD 4 = 0) Y (anio MOD 100 <> 0)) O (anio MOD 400 = 0)
FIN FUNCION

// Subalgoritmo: filtro de consistencia de calendario
FUNCION esFechaValida(d, m, a)
  SI (a < 1900) O (a > 2026)  ENTONCES RETORNAR FALSO
  SI (m < 1) O (m > 12)        ENTONCES RETORNAR FALSO
  SI (d < 1) O (d > 31)        ENTONCES RETORNAR FALSO
  SI (m EN {4, 6, 9, 11}) Y (d > 30) ENTONCES RETORNAR FALSO
  SI (m = 2) ENTONCES
        SI esBisiesto(a) ENTONCES
              SI (d > 29) ENTONCES RETORNAR FALSO
        SINO
              SI (d > 28) ENTONCES RETORNAR FALSO
        FIN SI
  FIN SI
  RETORNAR VERDADERO
FIN FUNCION
```

4.2. Construcción de la condición de ajuste

La parte central del ejercicio consistía en construir la condición que detecta si el cumpleaños ya ocurrió o aún no. El análisis lógico es el siguiente: la persona *sí cumplió* años si su mes de nacimiento es anterior al mes actual, o si su mes coincide con el actual pero su día de nacimiento ya pasó. La condición de ajuste es, por lo tanto, la negación de ese caso favorable, lo que se expresa directamente mediante el operador lógico OR:

```
SI (mesN > mesA) O ((mesN = mesA) Y (diaN > diaA)) ENTONCES
      edad <- edad - 1
```

La solución descarta deliberadamente la comparación directa entre días de meses distintos (por ejemplo, «¿el 30 de enero es posterior al 2 de marzo?»), que no es válida. La comparación de meses se hace primero y solo cuando estos coinciden se recurre al día. Además, la condición resultante es simétrica: si una persona nació el mismo día de la fecha de referencia, ya cumplió y la edad no se descuenta, porque la comparación estrictamente

mayor evita el descuento.

5. Desarrollo e implementación

5.1. Implementación en Python

El archivo proyecto_python/main.py implementa el algoritmo aprovechando las estructuras propias del lenguaje. El esqueleto (scaffolding) provisto define las funciones es_bisiesto(anio) y es_fecha_valida(d, m, a), y el bloque principal se protege con una estructura try/except ValueError que captura las entradas no numéricas.

```
def es_bisiesto(anio):
    return (anio % 4 == 0 and anio % 100 != 0) or (anio % 400 == 0)


def es_fecha_valida(d, m, a):
    if a < 1900 or a > 2026:
        return False

    if m < 1 or m > 12:
        return False

    if d < 1 or d > 31:
        return False

    if m in [4, 6, 9, 11] and d > 30:
        return False

    if m == 2:
        if es_bisiesto(a):
            if d > 29:
                return False
        else:
            if d > 28:
                return False

    return True


print("=====================================================")
print("    CALCULADORA EN PYTHON DE: Malek Cheheid    ")
print("=====================================================")

try:
    diaN = int(input("Ingrese Dia de nacimiento: "))
    mesN = int(input("Ingrese Mes de nacimiento: "))
    anioN = int(input("Ingrese Anio de nacimiento: "))

    diaA, mesA, anioA = 8, 7, 2026

    if not es_fecha_valida(diaN, mesN, anioN):
        print("[ERROR] La fecha ingresada no es valida.")
    else:
        edad = anioA - anioN

        if mesN > mesA or (mesN == mesA and diaN > diaA):
```

```
            edad = edad - 1

        print(f"\n[SISTEMA] Fecha de hoy: {diaA}/{mesA}/{anioA}")
        print(f"[SISTEMA] Edad calculada: {edad} anos.")

except ValueError:
    print("[ERROR] Deben ingresar numeros enteros.")

except EOFError:
    print("")
    print("[ERROR] No se pudo leer la entrada: estas ejecutando el archivo")
    print("con doble clic o con el boton Run, que no permiten escribir.")
    print("")
    print("Ejecutalo desde la TERMINAL de VS Code:")
    print("    cd proyecto_python")
    print("    python main.py")
```

5.2. Implementación en C++

El archivo proyecto_cpp/main.cpp traduce el mismo algoritmo al lenguaje estructurado. Las funciones se declaran antes de main() con su tipo de retorno explícito (bool), y la entrada se realiza en una única operación de flujo con tres operadores de extracción (cin >> diaN >> mesN >> anioN), lo que constituye una diferencia operativa relevante frente a las tres llamadas a input() separadas del código Python.

```
#include <iostream>
using namespace std;

bool esBisiesto(int anio)
{
    return (anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0);
}

bool esFechaValida(int d, int m, int a)
{
    if (a < 1900 || a > 2026) return false;

    if (m < 1 || m > 12) return false;

    if (d < 1 || d > 31) return false;

    if ((m == 4 || m == 6 || m == 9 || m == 11) && d > 30)
        return false;

    if (m == 2)
    {
        if (esBisiesto(a))
        {
            if (d > 29)
                return false;
        }
        else
        {
            if (d > 28)
                return false;
        }
    }
```

```
    return true;
}

int main()
{
    int diaN, mesN, anioN;

    int diaA = 8, mesA = 7, anioA = 2026;

    cout << "=====================================================" << endl;
    cout << "    CALCULADORA DE EDAD DE: Malek Cheheid    " << endl;
    cout << "=====================================================" << endl;

    cout << "Ingrese Dia, Mes y Anio de nacimiento (separados por espacios): ";
    cin >> diaN >> mesN >> anioN;

    if (!esFechaValida(diaN, mesN, anioN))
    {
        cout << "[ERROR] La fecha ingresada no existe en el calendario." << endl;
        return 1;
    }

    int edad = anioA - anioN;

    if (mesN > mesA || (mesN == mesA && diaN > diaA))
    {
        edad = edad - 1;
    }

    cout << "\n[SISTEMA] Fecha de hoy: "
         << diaA << "/" << mesA << "/" << anioA << endl;
    cout << "[SISTEMA] Edad calculada: " << edad << " anos." << endl;
    cout << "=====================================================" << endl;

    return 0;
}
```

6. Resultados

La batería de prueba se ejecutó sobre la implementación en Python, en la que se verificaron los diecinueve (19) casos. Dado que el equipo no disponía de compilador local de C++, la implementación en C++ se compiló y ejecutó en el entorno en línea OnlineGDB, donde se verificaron seis (6) de los casos más representativos. La tabla 1 presenta los resultados de la implementación en Python. La columna «Estado» indica si el programa produjo la edad esperada o si rechazó correctamente la entrada por no existir en el calendario.

Tabla 1

Resultados de los casos de prueba

N.º	Caso de prueba	Entrada	**Resultado**	**Estado**
1	Cumple años hoy mismo	8 / 7 / 2008	**18 años**	**Correcto**

N.º	Caso de prueba	Entrada	Resultado	Estado
2	Cumple años mañana	9 / 7 / 2008	**17 años (ajustado)**	**Correcto**
3	Cumple años ayer	7 / 7 / 2008	**18 años**	**Correcto**
4	Mes posterior, día anterior	7 / 8 / 2008	**17 años (ajustado)**	**Correcto**
5	Mes anterior	8 / 6 / 2008	**18 años**	**Correcto**
6	29 de febrero bisiesto	29 / 2 / 2008	**18 años**	**Correcto**
7	29 de febrero no bisiesto	29 / 2 / 2007	**Fecha inválida**	**Correcto**
8	Año 1900 no bisiesto	29 / 2 / 1900	**Fecha inválida**	**Correcto**
9	Año 2000 bisiesto secular	29 / 2 / 2000	**26 años**	**Correcto**
10	31 de abril	31 / 4 / 2000	**Fecha inválida**	**Correcto**
11	Mes 13	1 / 13 / 2000	**Fecha inválida**	**Correcto**
12	Día 0	0 / 1 / 2000	**Fecha inválida**	**Correcto**
13	Año 20900	1 / 1 / 20900	**Fecha inválida**	**Correcto**
14	Límite inferior del rango	1 / 1 / 1900	**126 años**	**Correcto**
15	Año bisiesto 2024	29 / 2 / 2024	**2 años**	**Correcto**
16	Nacimiento el día de hoy	8 / 7 / 2026	**0 años**	**Correcto**
17	Texto en vez de número	abc / 7 / 2008	**Error de tipo**	**Correcto**
18	Número decimal	8.5 / 7 / 2008	**Error de tipo**	**Correcto**
19	29 de febrero no bisiesto	29 / 2 / 1907	**Fecha inválida**	**Correcto**

Nota. La implementación en C++ produjo resultados idénticos a los de Python en la totalidad de los seis (6) casos verificados: 8/7/2008 → 18 años; 9/7/2008 → 17 años; 7/8/2008 → 17 años; 29/2/2000 → 26 años; 29/2/2007 → fecha inválida; y 31/4/2000 → fecha inválida. En todos los casos el compilador informó «Compiled Successfully» con código de salida 0. Dato relevante para la discusión: el caso 2 (nacimiento el día siguiente al de referencia) y el caso 4 (mes posterior con día anterior) demuestran que el ajuste de la edad no depende de comparar días de forma absoluta, sino de la comparación jeráquica de mes y luego de día.

7. Discusión: comparación de ambos lenguajes

Si bien el algoritmo es idéntico, la tabla 2 evidencia que cada lenguaje expresa la misma lógica con recursos distintos.

Tabla 2

Diferencias entre Python y C++ en la implementación

Aspecto	Python	C++
Declaración de variables	Dinámica; no se declara el tipo	Estática: int diaN, mesN, anioN;
Entrada por teclado	int(input(...)), tres llamadas	cin >> diaN >> mesN >> anioN;

Aspecto	Python	C++
Salida por pantalla	print(), con cadenas f	cout << ... << endl;
Conectores lógicos	and, or, not	&&, ||, !
Valores booleanos	True / False	true / false
Tipo de retorno	return sin tipo	bool esFechaValida(...)
Espacios de nombres	Módulos e importaciones	using namespace std;
Manejo de errores	try / except ValueError	Sin equivalente nativo
Fin de sentencia	No requiere punto y coma	Requiere punto y coma

7.1. Manejo de entradas erróneas

La diferencia más significativa observada se refiere al tratamiento de entradas no numéricas. La versión Python envuelve la lectura en un bloque try/except ValueError, que captura el error y muestra un mensaje controlado al usuario. En C++ no existe un mecanismo equivalente en la biblioteca estándar: si el usuario escribe una letra, el flujo de cin entra en estado de error, la variable queda sin asignar y el programa continúa ejecutándose con un valor indeterminado. Detectar esta situación en C++ exigiría consultar explícitamente el estado del flujo con cin.fail() o utilizar std::stoi dentro de un bloque try/catch, lo que incrementa el número de líneas de código. En definitiva, la gestión de errores es una ventaja de los lenguajes de alto nivel, que delegan buena parte de esa tarea al intérprete o al gestor de excepciones.

8. Conclusiones

Primera: el objetivo general se cumplió. El programa calcula la edad exacta de una persona y rechaza toda fecha que no exista en el calendario real, tanto en Python como en C++, con resultados idénticos entre ambos lenguajes en la totalidad de los casos verificados.

Segunda: la condición de ajuste resuelve correctamente los casos límite de cumpleaños. El caso crítico es el del nacimiento el mismo día de la fecha de referencia, que debe considerarse como cumpleaños ya ocurrido; la comparación estrictamente mayor garantiza este comportamiento sin casos especiales.

Tercera: la regla del año bisiesto debe implementarse en su totalidad. La inclusión de la cláusula secular (anio % 400 == 0) es indispensable: sin ella, el año 2000 sería rechazado como no bisiesto, y el programa fallaría en un caso que ocurre cada cuatro siglos.

Cuarta: la idea inicial se confirmó. El algoritmo es invariante, pero su expresión no lo es. Python favorece la legibilidad y la robustez (tipado dinámico, gestión integrada de errores), mientras que C++ favorece el control explícito y el rendimiento, a costa de más código y de mayor responsabilidad por parte del programador. Las dos implementaciones juntas ayudan a entender por qué se escribe algo tan distinto según el lenguaje.

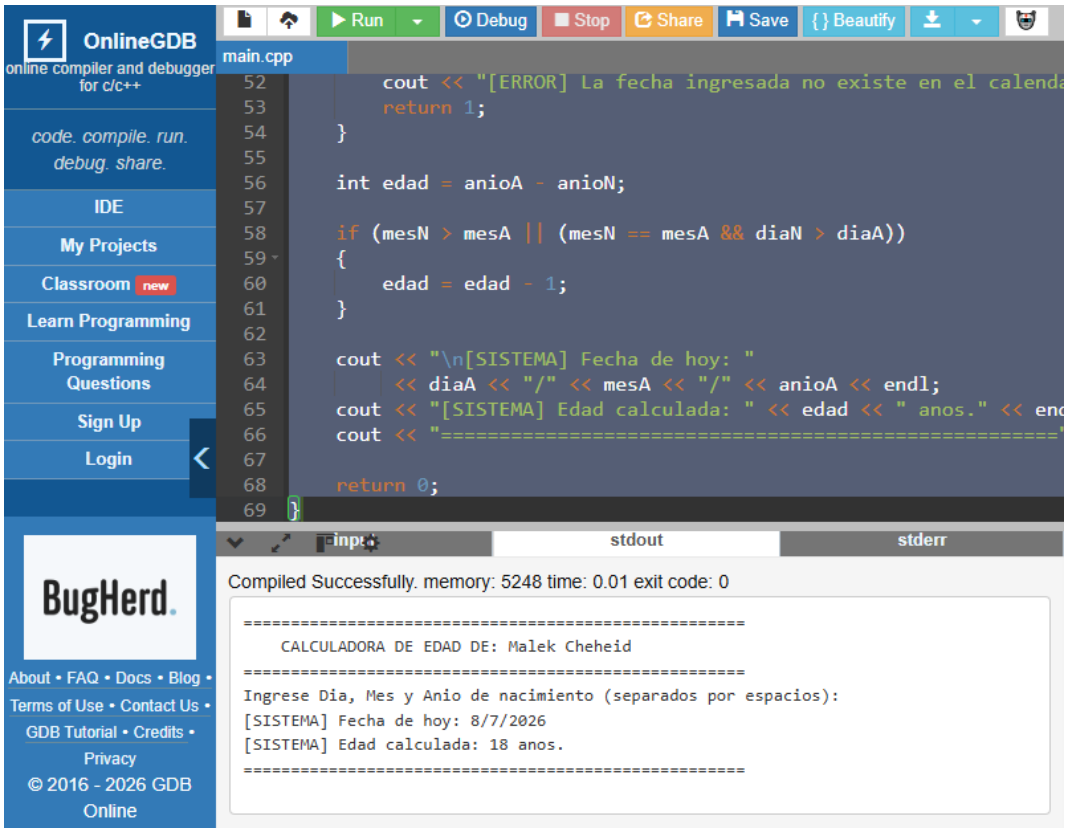
Quinta: como línea de trabajo futura se propone ampliar el programa para obtener la fecha actual desde el sistema en lugar de fijarla, y permitir el cálculo de la edad en meses y días transcurridos.

Anexo A. Evidencia de ejecución en OnlineGDB

El presente anexo documenta la ejecución del programa en C++ mediante el entorno en línea OnlineGDB, procedimiento exigido por la cátedra ante la imposibilidad de compilar localmente. Cada figura corresponde a un caso de prueba distinto y constituye evidencia del resultado obtenido.

Caso 1

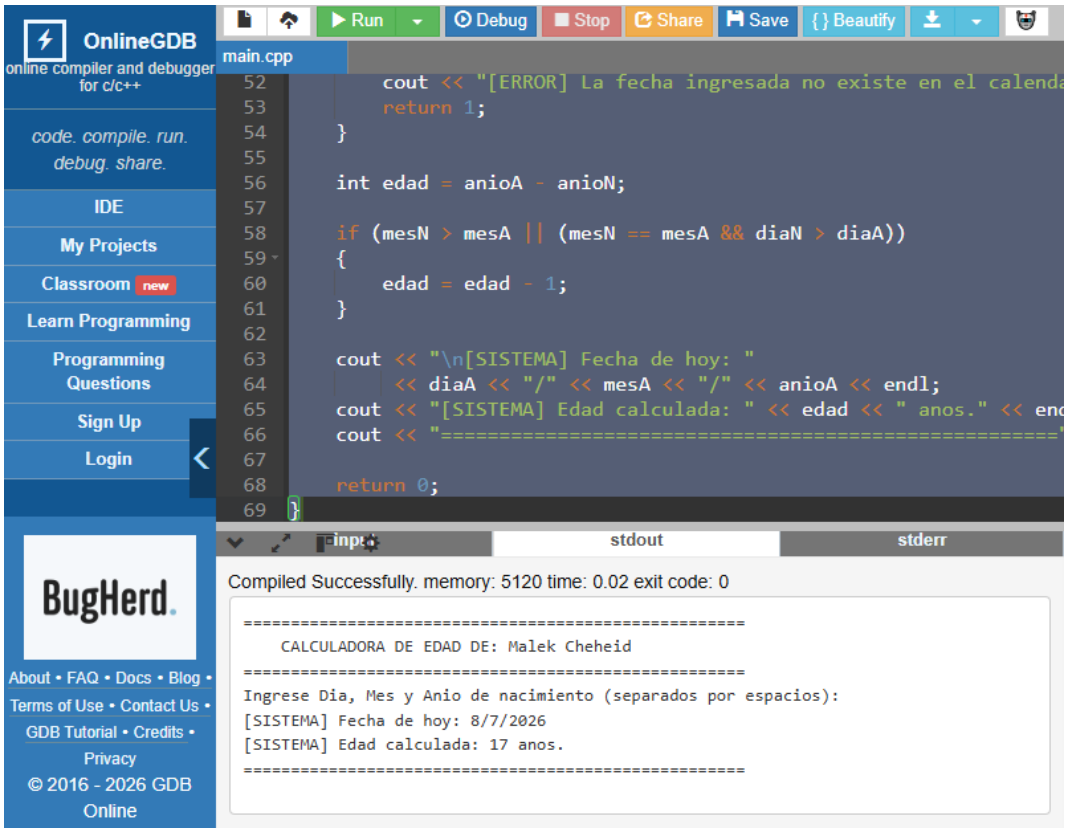
Caso base: la persona cumplió años el día 8 de julio de 2026.



Nota. Se ingreso 8 7 2008. El programa informa Compiled Successfully con código de salida 0 y calcula **18 años**. Al cumplirse el cumpleaños en la fecha de referencia, no se descuenta ningún año.

Caso 2

Caso crítico: la persona cumplió años mañana.



Nota. Se ingreso 9 7 2008. El resultado es **17 años**. Este caso es el que evidencia el funcionamiento de la condición completada: al ser posterior el día de nacimiento, la edad se descuenta en uno.

Caso 3

Rechazo de fechas inexistentes: 29 de febrero en un año no bisiesto.



Nota. Se ingreso 29 2 2007. El filtro de consistencia rechaza la fecha porque 2007 no es bisiesto y febrero solo tiene 28 días. El programa termina con código de salida 1.

Caso 4

Rechazo de meses de 30 días: 31 de abril.



Nota. Se ingreso 31 4 2000. La fecha se rechaza porque abril tiene 30 días. Este caso complementa al anterior y demuestra que el filtro opera sobre la cantidad de días de cada mes.